

MODUL 3



1 Acc
11/12 '25

Disusunoleh :

Nama : Ruhasdi

Nim : 24241016

Kelas : PTI A

Prodi : FSTT

Dosen Pembimbing :

Adam Bachtiar, S.Kom., M.MT.

PROGRAM STUDI PENDIDIKAN TEKNOLOGI
FAKULTAS SAINS DAN ILMU TERAPAN (FSTT)
UNIVERSITAS MANDALIKA MATARAM
2020/2025

Pratikum Percobaan - 1

```
PRINSIP OOP BAGIAN 2 - INHERITANCE (PEWARISAN) > Pratikum Percobaan - 1
1 class Kendaraan:
2     def __init__(self, nama):
3         self.nama = nama
4         print(f"Sebuah Kendaraan '{self.nama}' dibuat.")
5
6     def maju(self):
7         print(f"{self.nama} bergerak maju.")
8
9 class Mobil(Kendaraan):
10     def putar_AC(self):
11         print(f"{self.nama}: AC dinyalakan, sejuk!")
12
13 print("--- Membuat Object Mobil ---")
14 avanza = Mobil("Avanza Putih")
15 avanza.maju()
16 avanza.putar_AC()
```

Output

```
PS D:\Modul 3 PBO> & C:\Users\ASUS\AppData\Local\Programs\Python\Python313\python.
PBO/PRINSIP OOP BAGIAN 2 - INHERITANCE (PEWARISAN)/Pratikum Percobaan - 1
--- Membuat Object Mobil ---
Sebuah Kendaraan 'Avanza Putih' dibuat.
Avanza Putih bergerak maju.
Avanza Putih: AC dinyalakan, sejuk!
PS D:\Modul 3 PBO>
```

Penjelasan :

Baris 1: Mendefinisikan sebuah ~~class~~ class bernama kendaraan.

Baris 2: Mendefinisikan sebuah method kontruktor (`__init__`) yg akan dipanggil ketika objek dari class kendaraan dibuat. method ini menerima dua parameter: `self` (referensi objek yg dibuat) dan `nama` (nama kendaraan)

Baris 3: Mengset atribut `nama` dari objek dengan nilai yg diberikan.

Baris 4: Mencetak pesan bahwa sebuah kendaraan telah dibuat.

Baris 6: Mendefinisikan sebuah method bernama `maju` yg dapat dipanggil oleh objek dari class kendaraan.

Baris 7: Mencetak pesan bahwa kendaraan bergerak maju.

Baris 9: Mendefinisikan sebuah class bernama `Mobil` yg mewarisi dari class `Kendaraan`.

Baris 10: Sebuah method yg bernama Putar - AC yg dapat dipanggil oleh objek dari class mobil.

Baris 12: Menampilkan AC mobil menyala

Baris 13: menampilkan objek mobil yg dibuat

Baris 14: Membuat objek avanza dari class mobil dengan nama "Avanza putih"

Baris 15: memanggil method maju dari objek avanza.

Baris 16: memanggil method putar - AC dari objek avanza

Penjelasan tambahan:

1. class Parent = kendaraan
- class Child = mobil.

2. - kendaraan = ("sepeda motor").

3. Tidak bisa mengakses method Putar - AC karena method Putar - AC hanya dapat diakses di dalam class mobil.

4. Bisa mengakses method maju () karena class mobil mewarisi method maju dari class kendaraan

5. Tidak dibuat konstruktor karena class mobil mewarisi konstruktor dari class kendaraan.

Pratikum Percobaan - 2

```

PRATIKUM OOP BAGIAN 2 - INHERITANCE (PENGARISAN) 1 • Pratikum Percobaan - 2 1..
1 class Kendaraan:
2     def __init__(self, nama):
3         print("-> (Parent __init__ dipanggil)")
4         self.nama = nama
5
6 class Mobil(Kendaraan):
7     def __init__(self, nama, jumlah_pintu):
8         print("-> (Child __init__ dipanggil)")
9
10        super().__init__(nama)
11
12        self.jumlah_pintu = jumlah_pintu
13        print(f"-> Mobil {self.nama} dengan {self.jumlah_pintu} pintu")
14
15 xenia = Mobil("Xenia Hitam", 4)
16 print(f"Nama object: {xenia.nama}")
    
```

Output :

```

PS D:\Modul 3 PBO> & C:\Users\ASUS\AppData\Local\Programs\Python\Python313\python
PRATIKUM OOP BAGIAN 2 - INHERITANCE (PENGARISAN)/Pratikum Percobaan - 2
-> (Child __init__ dipanggil)
-> (Parent __init__ dipanggil)
-> Mobil Xenia Hitam dengan 4 pintu dibuat.
Nama object: Xenia Hitam
PS D:\Modul 3 PBO>
    
```

Penjelasan:

- Baris 1: mendefinisikan sebuah class bernama kendaraan.
- Baris 2: mendefinisikan sebuah method konstruktor (`__init__`) yg akan dipanggil ketika objek dari class kendaraan di ~~buat~~ dibuat.
- Baris 3: mencetak pesan bahwa konstruktor dari class kendaraan di panggil.
- Baris 4: mengset atribut nama dari objek dengan nilai yg diberikan.
- Baris 6: mendefinisikan sebuah class bernama mobil yg mewarisi dari class kendaraan.
- Baris 7: mendefinisikan sebuah method konstruktor (`__init__`) yg akan di panggil ketika objek dari class mobil dibuat.
- Baris 8: mencetak pesan bahwa konstruktor dari class mobil dipanggil.
- Baris 10: memanggil konstruktor dari kendaraan class dengan parameter nama. Ini digunakan untuk menginisialisasi atribut nama dari objek.

Baris 12: mengakses attribute jumlah_pintu dan objek dengan nilai yg diberikan.

Baris 13: Menampilkan mobil telah dibuat

Baris 15: Membuat objek Xenia dari class mobil dengan nama "Xenia hitam" dan jumlah pintu 4

Baris 16: Menampilkan nama objek Xenia.

Pengelasan tambahan:

1. Nama konstruktor dari class kendaraan tidak akan di panggil karena atribut nama tidak di definisikan
2. Karena konstruktor dari class kendaraan tidak di panggil, sehingga atribut nama tidak di definisikan
3. Super Lisa di pakai child memanggil method parent. Super digunakan untuk memanggil method parent yg telah di override oleh class child.

Praktikum Perencanaan – 3

```

1 class Hero:
2     def __init__(self, name, health):
3         self.name = name
4         self.health = health
5
6     def showInfo(self):
7         print("showInfo of class Hero")
8         print("Name: {} Health: {}".format(
9             self.name,
10            self.health))
11
12
13 class Hero_Intelligent(Hero):
14     def __init__(self, name):
15         super().__init__(name, 100)
16
17     def showInfo(self):
18         print("showInfo of subclass Hero_Intelligent")
19         print("Hero: Intelligent, Name: {}".format(
20             self.name,
21             self.health))
22
23
24 class Hero_Strong(Hero):
25     def __init__(self, name):
26         super().__init__(name, 200)
27
28
29 lha = Hero_Intelligent("lha")
30 sh = Hero_Strong("sh")
31
32 lha.showInfo()

```

Output:

```
PS D:\Modul 3 PBO> & C:\Users\ASUS\AppData\Local\Programs\Python\
Python38\python.exe C:\Users\ASUS\AppData\Local\Programs\Python\Python38\
PROTPRINSIP OOP BAGIAN 2 - INHERITANCE (P44815AN)/Praktikum Per
pythonInfo di subclass Hero_Intelligent
Info
    Tipe: intelligent,
    health: 100
PS D:\Modul 3 PBO>
```

Penjelasan

Basis 1: mendefinisikan sebuah class bernama hero

Baris 2: mendefinisikan sebuah method konstructor (-vnt-) yg akan di panggil ketika objek dari class hero di buat.

Baris 3 : mengakses atribut name dari objek dengan nilai yg diberikan

Baris 4: Mengakses atribut health dari objek dengan nilai 99 diberikan

Baris 6 : Mendefinisikan sebuah method bernama Show Info yg dapat dipanggil oleh objek dari class here.

Baris 7: Menampilkan method show info di Panggil dari class hero

Baris 8-10: menampilkan hero, termasuk nama dan health.

Baris 11: Sebuah class bernama hero - intelligent yg mewarisi class hero.

Baris 15: Sebuah method konstruktor yg akan di Panggil ketika objek class - intelligent dibuat.

Baris 16: memanggil konstruktor dari class hero dengan parameter nama dan 100 sebagai health hero

Baris 18: Sebuah method bernama show info yg dapat di Panggil oleh objek class hero - intelligent.

Baris 19: Menampilkan method show info di Panggil dari class hero - intelligent.

Baris 20-22: menampilkan hero, termasuk nama, tipe, dan health.

Baris 25: Sebuah class bernama hero - strength yg mewarisi hero class.

Baris 26: Sebuah method konstruktor yg akan di Panggil ketika objek dibuat.

Baris 27: memanggil konstruktor dari class hero dengan parameter nama dan 200 sebagai health hero.

Baris 29: membuat objek lina dari class hero - intelligent dengan nama "lina"

Baris 30: membuat objek ake dari class hero - strength dengan nama "axe".

Baris 32: memanggil method show info dari objek lina.

Penjelasan tambahan.

1. Method Show Info di override oleh class hero_intelligent.
2. Kita dapat menggunakan super.Show Info pada child class.

Pratikum Percobaan - 4

```
PRINSIP OOP BAGIAN 2 - INHERITANCE (PEWARISAN) > Pratikum Percobaan - 4 >
1 class Ayah:
2     def __init__(self, nama_ayah):
3         self.nama_ayah = nama_ayah
4
5     def bekerja(self):
6         print(f"{self.nama_ayah} sedang bekerja.")
7
8 class Ibu:
9     def __init__(self, nama_ibu):
10        self.nama_ibu = nama_ibu
11
12    def memasak(self):
13        print(f"{self.nama_ibu} sedang memasak.")
14
15 class Anak(Ayah, Ibu):
16     def __init__(self, nama_anak, nama_ayah, nama_ibu):
17         Ayah.__init__(self, nama_ayah)
18         Ibu.__init__(self, nama_ibu)
19         self.nama_anak = nama_anak
20
21     def bermain(self):
22         print(f"{self.nama_anak} sedang bermain.")
23
24 budi = Anak("Budi", "Hendra (Ayah)", "Wati (Ibu)")
25
26 budi.bekerja()
27 budi.masak()
28 budi.bermain()
```

Output:

```
PS D:\Modul 3 PBO> & C:/Users/ASUS/AppData/Local/Programs/Python/Python313/python.
PRG/PRINSIP OOP BAGIAN 2 - INHERITANCE (PEWARISAN)/Pratikum Percobaan - 4
Hendra (Ayah) sedang bekerja.
Wati (Ibu) sedang memasak.
Budi sedang bermain.
PS D:\Modul 3 PBO>
```

Penjelasan:

- Baris 1: Men definisikan sebuah class bernama ayah
- Baris 2: method konstruktor (`__init__`) yg akan di panggil ketika objek ayah sedang dibuat.
- Baris 3: Mengakset atribut `nama_ayah` dari objek dengan nilai yg diberikan.
- Baris 5: Men definisikan sebuah method bernama `bekerja` yg dapat dipanggil oleh objek dari class ayah

Baris 6: Menampilkan ayah sedang bekerja.

Baris 8: Membuat class bernama ibu.

Baris 9: Sebuah method kontruktor yg akan dipanggil ketika objek dari class ibu dibuat

Baris 10: Mengakses atribute ibu nama_ibu dari objek dengan nilai yg diberikan

Baris 12: Sebuah method bernama memasak yg dapat dipanggil oleh objek dari class ibu.

Baris 13: Menampilkan ibu sedang memasak.

Baris 15: Sebuah class bernama anak yg mewarisi dari class ayah dan ibu.

Baris 16: Sebuah method kontruktor akan dipanggil - ketika objek dari class anak dibuat

Baris 17: Memanggil konstruktor dari class ayah dengan parameter nama_ayah.

Baris 18: Memanggil konstruktor dari class ibu dengan parameter nama_ibu

Baris 19: Mengakses atribut nama_anak dari objek dengan nilai yg diberikan

Baris 20: Sebuah method bermain dipanggil oleh objek dari class anak.

Baris 22: Menampilkan anak sedang bermain.

Baris 24: Membuat objek budi dari class anak dengan nama "Budi", ayah "Hendra", ibu "Wati".

Baris 26: Memanggil method bekerja oleh objek budi

Baris 27: Memanggil method memasak oleh objek budi

Baris 28: Memanggil method bermain oleh objek budi.

Penjelasan tambahan:

• maka versi yg dipakai anak adalah versi class ayah karena ayah adalah versi pertama kali diwarisi oleh anak. jika menggunakan versi ibu class, dapat memanggil method pengasah dari class ibu secara langsung.

```

1 class Pegawai:
2     def __init__(self, nama, nip, gaji_pokok):
3         self.nama = nama
4         self.nip = nip
5         self.__gaji_pokok = gaji_pokok  # Atribut private untuk enkapsulasi
6
7     def get_gaji_pokok(self):
8         return self.__gaji_pokok
9
10    def hitung_bonus(self):
11        # Method ini akan di-override oleh child class
12        return 0
13
14    def get_gaji_total(self):
15        return self.__gaji_pokok + self.hitung_bonus()
16
17    def tampilkan_info(self):
18        print(f>Nama: {self.nama}, NIP: {self.nip}")
19        print(f>Gaji Pokok: Rp {self.__gaji_pokok:,}")
20
21 class Manager(Pegawai):
22     def __init__(self, nama, nip, gaji_pokok, tunjangan_jabatan):
23         super().__init__(nama, nip, gaji_pokok)
24         self.tunjangan_jabatan = tunjangan_jabatan
25
26     def hitung_bonus(self):
27        # Bonus manager: 15% dari gaji pokok
28        return 0.15 * self.get_gaji_pokok()
29
30    def tampilkan_info(self):
31        print("--- Info Manager ---")
32        super().tampilkan_info()

```

TUGAS PRAKTIKUM MODUL 3

```
1 class Pegawai:
2     def __init__(self, nama, nip, gaji_pokok):
3         self.nama = nama
4         self.nip = nip
5         self.__gaji_pokok = gaji_pokok + 1000000
6
7     def get_gaji_pokok(self):
8         return self.__gaji_pokok
9
10    def hitung_bonus(self):
11        # Method ini akan di-override oleh child class
12        return 0
13
14    def get_gaji_total(self):
15        return self.__gaji_pokok + self.hitung_bonus()
16
17    def tampilkan_info(self):
18        print(f'Nama: {self.nama}, NIP: {self.nip}')
19        print(f'Gaji Pokok: Rp {self.__gaji_pokok:,}')
20
21 class Manajer(Pegawai):
22     def __init__(self, nama, nip, gaji_pokok, tunjangan_jabatan):
23         super().__init__(nama, nip, gaji_pokok)
24         self.tunjangan_jabatan = tunjangan_jabatan
25
26     def hitung_bonus(self):
27        # Bonus Manajer: 10% dari gaji pokok
28        return 0.1 * self.get_gaji_pokok()
29
30     def tampilkan_info(self):
31        print('--- Info Manajer ---')
32        super().tampilkan_info()
```

```

        print(f"tunjangan: Rp {self.tunjangan_jabatan:.}")
        print(f"Gaji total Manager: Rp {self.get_gaji_total():.}")

class StaffTeknis(Pegawai):
    def __init__(self, nama, nip, gaji_pokok, jumlah_proyek):
        super().__init__(nama, nip, gaji_pokok)
        self.jumlah_proyek = jumlah_proyek

    def hitung_bonus(self):
        # Bonus staff teknis: 500.000 dikali jumlah proyek
        return 500000 * self.jumlah_proyek

    def tampilkan_info(self):
        print("--- Info Staff Teknis ---")
        super().tampilkan_info()
        print(f"Jumlah Proyek: {self.jumlah_proyek}")
        print(f"Gaji total Staff: Rp {self.get_gaji_total():.}")

# Contoh penggunaan
if __name__ == "__main__":
    # Membuat objek Manager
    manager = Manager("Budi Hartono", "M-001", 10000000, 500000)
    manager.tampilkan_info()
    print("\n=====")

    # Membuat objek StaffTeknis
    staff = StaffTeknis("Susi Susanti", "S-001", 6000000, 3)
    staff.tampilkan_info()
    print("\n=====")

```

```

# Tes Keamanan (Encapsulasi)
print("--- Tes Keamanan (Encapsulasi) ---")
try:
    print(staff.__gaji_pokok) # Mencoba mengakses atribut private
except AttributeError as e:
    print(f"ERROR: {e}")
    print("-> TIDAK BISA diakses langsung dari luar!")
print(f"Gaji total Susi (tetap): Rp {staff.get_gaji_total():.}")

```

[illegible]

```
Info Staff (M001)
Name: Suci Santia, NIP: 5-001
Gaji Pokok: Rp 6,000,000
Jumlah Proyek: 3
Gaji Total Staff: Rp 7,500,000

-----
Info Karyawan (Incapsulasi)
ERROR: 'StaffM001' object has no attribute '__gaji_pokok'
+ TILAK BISA diakses langsung dari luar!
Gaji Total Suci (tutup): Rp 7,500,000
```

Baris 1: membuat class bernama pegawai
Baris 2: membuat sebuah method konstruktor yg akan di panggil ketika objek dari class pegawai di buat
Baris 3: mengakses atribut nama dari objek dari nilai yg diberikan
Baris 4: mengakses atribut umur dari objek dari nilai yg diberikan
Baris 5: mengakses atribut gaji pokok dari objek nilai yg diberikan
Baris 6: membuat sebuah method getter untuk mendapatkan nilai gaji pokok dari objek
Baris 7: mengembalikan nilai gaji pokok dari objek
Baris 8: membuat fungsi hitung bonus di dalam class pegawai
Baris 9: Fungsi hitung bonus akan diganti override oleh class child
Baris 10: mengembalikan nilai sebagai default
Baris 11: membuat fungsi bernama get_gaji_total di dalam class pegawai

Baris 16: menghitung gaji total dengan menjumlahkan gaji pokok dan bonus.

Baris 17: membuat fungsi bernama tampil_info di dalam class Pegawai

Baris 18: menampilkan rupiah nama pegawai.

Baris 19: menampilkan gaji pokok pegawai.

Baris 21: membuat class baru bernama manager yg memwarisi dari class Pegawai

Baris 22: membuat method konstruktor untuk class manager untuk objek (nama, nip, gaji_pokok dan tunjangan - jabatan).

Baris 23: menjalankan class konstruktor untuk pegawai dan memberikan nama, nip, dan gaji_pokok.

Baris 24: menampilkan tunjangan jabatan yg diberikan ke dalam objek manager.

Baris 26: mengganti override fungsi hitung_bonus yg ada di class Pegawai.

Baris 27: menghitung bonus manager sebesar 15% dari gaji pokok.

Baris 30: mengganti override fungsi tampilan_info yg ada di class Pegawai.

Baris 31: menampilkan info manager.

Baris 32: menampilkan fungsi tampilan_info dari class Pegawai

Baris 33: menampilkan tunjangan_jabatan manager

Baris 34: menampilkan gaji_total manager

Baris 36: membuat class baru bernama staff teknis.

Baris 37: membuat konstruktor class staff teknis.

Baris 38: Menjalankan konstruktor dari class pegawai dan
membetikan nama, nip dan gaji pokok.

Baris 39: Mengimpor jumlah proyek yg kita berikan ke
dalam objek Staff teknis.

Baris 41: Mengganti Override fungsi hitung Bonus yg ada di
class pegawai

Baris 42: Menghitung Bonus Staff teknis sebesar 500000 di
kali jumlah proyek.

Baris 44: Mengganti fungsi tampilan_intro yg ada
di dalam class pegawai

Baris 45: Menampilkan informasi Staff teknis

Baris 46: Menjalankan fungsi tampilan_intro dari
class pegawai.

Baris 47: Menampilkan jumlah proyek Staff teknis.

Baris 48: Menampilkan gaji total Staff teknis.

Baris 50: membuat objek dari class manager dan
Staff teknis.

Baris 54: manager (budi Hartono).